The artifact I selected for this milestone is my CS-330 3D Scene Project from Computational Graphics and Visualization. The project was originally created during CS-330 and involved building and rendering a fully textured 3D scene using C++, OpenGL, GLFW, GLEW, and GLSL shaders. The scene includes multiple objects, textures, lighting effects, camera movement, and both perspective and orthographic viewing modes. Some of the objects included in the scene are a coffee mug, flashlight, dog tags, coin, and a textured tabletop surface. The project also allows the user to navigate around the environment using keyboard and mouse controls. I selected this artifact because it demonstrates several different technical skills at once. The project required an understanding of graphics programming, object transformations, lighting, texture mapping, and scene management. It also gave me an opportunity to work with a larger codebase than I had in some of my earlier courses. Since I currently work in IT as a Senior Desktop Support Analyst, I thought this project was a good way to show growth beyond my normal day-to-day responsibilities and demonstrate software development and graphics programming skills that I have developed throughout the Computer Science program.

I chose this artifact for the Software Design and Engineering category because it gave me a strong opportunity to improve the structure and organization of an already functional project. Originally, most of the scene rendering logic existed inside one very large RenderScene() method. While the project worked correctly, having nearly all rendering operations inside one method made the code harder to read, harder to maintain, and more difficult to expand later. For this enhancement, I refactored the rendering logic into multiple smaller helper functions based on each major object or scene component. Instead of keeping all rendering logic together in one section, the project now contains separate rendering methods such as RenderTable(), RenderMug(), RenderFlashlight(), RenderCoin(), RenderDogTags(), and RenderDogTagChain().

I also moved the lighting configuration into its own SetupSceneLighting() method. This significantly reduced clutter inside the main RenderScene() function and made the overall structure of the project easier to follow.

One of the biggest improvements from this enhancement was maintainability. If I want to change the mug, flashlight, or another object later, I no longer need to search through one large rendering function to find the correct section of code. Separating the rendering responsibilities into their own functions also makes future enhancements easier because each section can be modified independently without affecting unrelated parts of the scene. Overall, the refactor helped make the project look and feel more like a professionally organized graphics application instead of a single large student assignment file.

This enhancement helped me continue working toward several course outcomes within the Computer Science program. One outcome that connects strongly to this project is the ability to design and evaluate computing solutions using software engineering practices and standards. Through this enhancement, I improved the organization, readability, and maintainability of the project by restructuring the rendering logic into smaller reusable components. This experience helped reinforce the importance of modular design and separation of responsibilities when working on larger projects. Another outcome demonstrated through this enhancement is professional communication and technical documentation. As I worked through the refactor, I added comments and organized the code more clearly so that someone else reviewing the project could follow the structure more easily. This also helped me better understand how important clean and understandable code becomes as projects increase in size and complexity. I believe this enhancement also demonstrates growth in my ability to use industry-relevant tools and practices. Working with OpenGL, shader management, object transformations, and scene rendering already

required technical problem-solving, but the refactoring process pushed me further by making me think about software architecture and maintainability rather than simply making the project function correctly.

One of the biggest things I learned during this enhancement process was how quickly large rendering methods can become difficult to manage. Even though the original project worked, it became obvious while reviewing the code that keeping everything inside one RenderScene() function made the project harder to navigate and update. Breaking the rendering code into smaller methods made the project feel much more organized and easier to understand. The most challenging part of the enhancement was making sure I did not accidentally break transformations, textures, or lighting while moving sections of code into separate functions. Since many objects depended on specific transformations and texture settings, even small mistakes could cause objects to disappear, render incorrectly, or lose textures. Because of that, I tested the project after nearly every major change to make sure the scene still rendered properly before continuing.

Another challenge was making sure this enhancement stayed focused specifically on Software Design and Engineering without overlapping too heavily into the Algorithms and Data Structures or Databases categories that I plan to complete later. Since I am using the same artifact for all three categories, I had to be careful about keeping this milestone centered on organization, modularity, readability, and maintainability rather than optimization or external data management. Overall, this enhancement helped me better understand the importance of software organization and maintainable design in larger applications. It also showed me that writing working code is only one part of software development, while structuring code in a way that is readable and scalable is equally important.